Chapitre 0 — Avant tout | Partie A — Écrire des lettres | leçon 0.9.1 sur 153 | difficulté 1 / 10

0.9.1. Tout l'alphabet, avec une seule boucle — de base, ailleurs

Le 0.9 trois lignes plus bas : tout l'alphabet sur les lignes 3 et 4.

1. Ce qu'il faut comprendre

C'est le 0.9, avec un seul changement : la **ligne de départ** : `3 + i / 20` au lieu de `i / 20`. `i / 20` vaut 0 puis 1 ; en ajoutant 3, on obtient les lignes 3 puis 4.

Le reste ne bouge pas : A à T sur une ligne, U à Z sur la suivante.

C'est la version de base : une seule chose, à la deuxième place. Le 0.9.2 met les deux ensemble.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <reste>` → 0.8.4. #include <reste> — a % b
- `#include <diviser>` → 0.8.5. #include <diviser> — a / b
- `#include <ALPHABET>` → 0.1.4. ALPHABET — les lettres de la police
- `#include <poser>` → 0.1.3. La fonction poser() — poser une tuile

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **affectation (=)** — « x = 5; » : la boîte x prend la valeur 5. Ce n'est pas une égalité : c'est un rangement.
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **coordonnées** — Deux nombres qui disent où : la colonne (vers la droite) puis la ligne (vers le bas), à partir de (0, 0) en haut à gauche.
- **for** — Une boucle qui compte : « for (uint8_t i = 0; i < 5; i++) » fait 5 tours, i valant 0, 1, 2, 3 puis 4.
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **incrémenter (++)** — « x++ » ou « x += 1 » : x augmente de 1. « x-- » : x diminue de 1.
- **indice** — Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **octet** — Un nombre de 0 à 255 : la plus petite boîte de la mémoire. Une cartouche en a 32 768.
- **opérateurs de calcul** — + plus, - moins, * fois, / divisé par (sans virgule : 7 / 2 donne 3), % le reste de la division (7 % 2 donne 1).
- **opérateurs de comparaison** — == égal, != différent, < plus petit, > plus grand, <= plus petit ou égal, >= plus grand ou égal. Le résultat est vrai ou faux.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **poser()** — Met une tuile dans le décor, sur une case de la grille de 20 × 18 (32 × 32 en tout, avec la partie cachée).
- **uint8_t** — Le genre de nombre de cette console : un entier de 0 à 255 (un octet). Au-delà, il repart à 0.
- **variable** — Une boîte avec un nom, dans la mémoire, qui retient un nombre qu'on peut lire et changer.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`#include <reste>  // a % b, sauf par 1, 2, 4, 8, 16… écrits en clair`	Demande à la console « reste » (ce que c'est : a % b, sauf par 1, 2, 4, 8, 16… écrits en clair). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « a % b, sauf par 1, 2, 4, 8, 16… écrits en clair ».)
`#include <diviser>  // a / b, sauf par 1, 2, 4, 8, 16… écrits en clair`	Demande à la console « diviser » (ce que c'est : a / b, sauf par 1, 2, 4, 8, 16… écrits en clair). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « a / b, sauf par 1, 2, 4, 8, 16… écrits en clair ».)
`#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A`	Demande à la console « ALPHABET » (ce que c'est : les lettres de la police : ALPHABET[0] est le A). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « les lettres de la police : ALPHABET[0] est le A ».)
`#include <poser>  // pose une tuile sur une case du fond`	Demande à la console « poser » (ce que c'est : pose une tuile sur une case du fond). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « pose une tuile sur une case du fond ».)
`int main() {  // Le programme commence ici.`	Le début du programme : la console commence ici, par la première ligne qui suit. (Le commentaire le dit : « Le programme commence ici. ».)
`// Le changement : les lignes 0 et 1 → 3 et 4.`	Un commentaire : une phrase pour celui qui lit (« Le changement : les lignes 0 et 1 → 3 et 4. »). La console l'ignore.
`// for : i part de 0, et gagne 1 à chaque tour tant que i < 26 (la taille de ALPHABET).`	Un commentaire : une phrase pour celui qui lit (« for : i part de 0, et gagne 1 à chaque tour tant que i < 26 (la taille de ALPHABET). »). La console l'ignore.
`for (uint8_t i = 0; i < sizeof(ALPHABET); i++) {`	Une boucle qui compte : i part de 0 ; on continue tant que i est plus petit que sizeof(ALPHABET) ; et après chaque tour, i augmente de 1.
`poser(i % 20, 3 + i / 20, ALPHABET[i]);  // 3 + i / 20 : la ligne 3, puis 4`	Appel de la fonction poser de la console, avec les arguments i % 20, 3 + i / 20, ALPHABET[i]. Ce qu'elle fait : pose une tuile sur la grille du fond — un nom de Tuile, ou un numéro. Un 4e argument est une TAILLE : poser(2, 3, MUR, 5) dessine MUR cinq fois plus grand dans le fond (colonne et ligne en clair). (Le commentaire le dit : « 3 + i / 20 : la ligne 3, puis 4 ».)
`// i % 20 = le reste par 20 : la colonne`	Un commentaire : une phrase pour celui qui lit (« i % 20 = le reste par 20 : la colonne »). La console l'ignore.
`//  (0 à 19, puis de nouveau 0 à 5).`	Un commentaire : une phrase pour celui qui lit (« (0 à 19, puis de nouveau 0 à 5). »). La console l'ignore.
`// i / 20 = 0 pour A à T, 1 pour U à Z.`	Un commentaire : une phrase pour celui qui lit (« i / 20 = 0 pour A à T, 1 pour U à Z. »). La console l'ignore.
`// Ex. : i = 22 (le W) → colonne 2, ligne 4.`	Un commentaire : une phrase pour celui qui lit (« Ex. : i = 22 (le W) → colonne 2, ligne 4. »). La console l'ignore.
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`while (true) {  // La boucle de jeu : pour toujours...`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image(). (Le commentaire le dit : « La boucle de jeu : pour toujours... ».)
`image();  // ... attendre l'image suivante.`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu. (Le commentaire le dit : « ... attendre l'image suivante. ».)
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **940 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage        49 o  ← la console, à l'allumage  — la mise en route de la console
  main()           66 o  ← démarrage  — ta fonction
  VBlank            8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage    28 o  ← image()  — attend l'image suivante
  AttendreVBlank   16 o  ← démarrage  — attend que l'écran soit libre
  AdresseCase      10 o  ← poser()  — trouve l'adresse d'une case de l'écran
  Diviser          17 o  ← poser()  — divise — le processeur ne sait pas le faire
  Reste            10 o  ← poser()  — le reste d'une division
  CopierTuiles     18 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte     14 o  ← démarrage  — vide les deux cartes de l'écran
  Tuiles          704 o  ← CopierTuiles  — la police entière
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, EffacerCases, EcrireTexte, EcrireNombre,
Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau,
LireManette, DonneesTransfert, Notes
Fonctions du langage appelées :
  poser() × 1    41 o sur place, dans main()  + AdresseCase (10 o), Diviser (17 o), Reste (10 o)
  image() × 1     3 o sur place, dans main()  + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <reste>     // a % b, sauf par 1, 2, 4, 8, 16… écrits en clair
#include <diviser>   // a / b, sauf par 1, 2, 4, 8, 16… écrits en clair
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
#include <poser>      // pose une tuile sur une case du fond
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

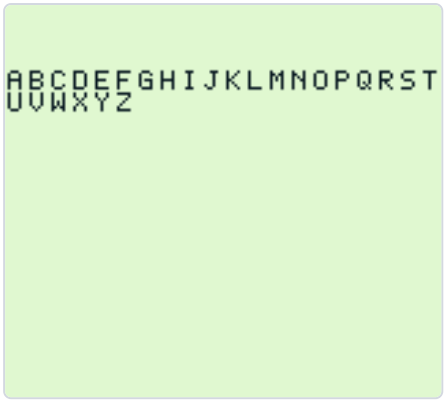
```
int main() {                                // Le programme commence ici.
  // Le changement : les lignes 0 et 1 → 3 et 4.
  // for : i part de 0, et gagne 1 à chaque tour tant que i < 26 (la taille de ALPHABET).
  for (uint8_t i = 0; i < sizeof(ALPHABET); i++) {
    poser(i % 20, 3 + i / 20, ALPHABET[i]);   // 3 + i / 20 : la ligne 3, puis 4
                                              // i % 20 = le reste par 20 : la colonne
                                              //   (0 à 19, puis de nouveau 0 à 5).
                                              // i / 20 = 0 pour A à T, 1 pour U à Z.
                                              // Ex. : i = 22 (le W) → colonne 2, ligne 4.
  }

  while (true) {                              // La boucle de jeu : pour toujours...
    image();                                  // ... attendre l'image suivante.
  }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

> Tout l'alphabet sur les lignes 3 et 4.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ ligne 3 : A à T
- ✓ ligne 4 : U à Z

Ensuite : **0.9.2. Tout l'alphabet, avec une seule boucle — doublé, deux positions** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++